Spa Wellness

Software Requirements Specification

Project:	Spa Wellness Web Platform
Stack:	MERN (MongoDB, Express, React, Node.js)
Document Type:	Software Requirements Specification
Version:	1.0
Owned By:	Ridhima

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Spa Wellness** application. It is intended for developers, stakeholders, and testers to understand the functionality and constraints of the system in detail.

1.2 Scope

Spa Wellness is a comprehensive web platform for a spa and wellness center. It facilitates service discovery, appointment booking, and administrative management. The project uses the **MERN** stack — MongoDB, Express.js, React.js, and Node.js.

2. Overall Description

2.1 Product Perspective

The system is a distributed application with a clear separation between the frontend (React / Vite) and the backend (Node.js / Express / MongoDB). Both tiers communicate through a RESTful API.

2.2 Product Functions

- **User Management**: Registration, login, and profile management.
- **Service Catalog**: Displaying all available spa services to users.
- **Booking System**: Selecting services, dates, and times for appointments.
- **Administrative Control**: Managing user bookings — approve or reject.
- **Feedback / Reviews**: Users can submit feedback on completed services.

2.3 User Classes and Characteristics

- **Guest**: Unauthenticated visitors who can browse the service catalog.
- **Customer (User)**: Authenticated users who can book and track appointments.
- **Admin**: Staff members who manage business operations and bookings.

3. System Features

3.1 Authentication & Authorization

- **Registration**: Users can create an account using their name, email, and a secure password.
- **Login**: Secure login using JWT (JSON Web Tokens).
- **Role-Based Access**: Restricts certain pages (e.g., Admin Dashboard) to users with the 'admin' role.

3.2 Booking Module

- **Create Booking**: Users can choose a service, date, and time slot.

- **Status Tracking**: Bookings carry one of three statuses: Pending, Accepted, or Rejected.
- **Validation**: All required fields must be provided before submission is accepted.

Status	Description
Pending	Default state after booking creation; awaiting admin review.
Accepted	Admin has confirmed the appointment.
Rejected	Admin has declined the booking request.

3.3 Admin Module

- **Dashboard**: A centralised view for admins to see all incoming booking requests.
- **Booking Management**: Tools to change the status of any booking (Pending → Accepted / Rejected).

4. External Interface Requirements

4.1 User Interfaces

The frontend is built with **React** and **Tailwind CSS** to deliver a modern, responsive user interface. The design follows a clean, wellness-focused aesthetic with intuitive navigation suitable for all user classes.

4.2 Hardware Interfaces

No special hardware interfaces are required. Access is provided through any standard modern web browser on desktop or mobile devices.

4.3 Software Interfaces

- **Database**: MongoDB (NoSQL) is used for flexible, schema-less data storage hosted via MongoDB Atlas.
- **Backend API**: RESTful services are built with Express.js and communicate using JSON payloads.

5. Non-Functional Requirements

5.1 Security

- **Password Encryption**: All passwords are hashed using bcryptjs before storage.
- **JWT Middleware**: Secure API endpoints verify tokens via Express middleware.
- **CORS Policy**: Cross-Origin Resource Sharing is configured to permit only trusted origins.

5.2 Performance

- **Fast Builds**: Vite's optimised build process ensures minimal page-load times.
- **Efficient Queries**: Mongoose ODM is used to write efficient, indexed MongoDB queries.

5.3 Scalability

The backend follows a modular architecture — **Models**, **Controllers**, and **Routes** are kept separate — which allows new features to be added with minimal impact on existing code. MongoDB Atlas enables horizontal scaling as demand grows.

6. Technical Stack

Layer	Technology	Purpose
Frontend	React.js, Vite, Tailwind CSS, React Router	UI rendering, routing, and styling
Backend	Node.js, Express.js	REST API server and business logic
Database	MongoDB Atlas (via Mongoose ODM)	Persistent data storage
Authentication	JWT, Bcrypt.js	Secure login and password hashing
Dev Tooling	Vite	Hot-reload development and optimised builds